

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 8

The Same Calls, Inside Your Own OS

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. run, inside xv6 **2**

2 Part 2. pingpong **4**

Conclusions **5**

Purpose of the work

In Lab 7 I ran `fork`, `exec` and `wait` on Linux. The purpose of this lab is to run the same program inside xv6, the small operating system built during this semester, and to see that the calls are the same; and then to make two processes talk to each other through pipes with a program called `pingpong`.

The work was done in my `~/xv6-riscv` tree (`mit-pdos/xv6-riscv`, commit `06aad25`), cross-compiled with `riscv64-linux-gnu-gcc` 16.2.1 and run on QEMU 11.1.1. Every new program is a file `user/NAME.c` plus one line `$U/_NAME\` in `UPROGS`, and every run started from a fresh disk image.

1 Part 1. run, inside xv6

The program performs the same three steps as `run.c` in Lab 7, but there is no glibc in xv6, only the system calls of the kernel I compiled. The child becomes `echo`.

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                    // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                        // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                // step 2: become echo
    printf("exec failed\n");           // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                       // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Figure 1.1 – user/run.c

```
$ grep -n "_run\|_pingpong" Makefile
157:    $U/_run\
158:    $U/_pingpong\
```

Figure 1.2 – The UPROGS lines for run and pingpong

```
$ make qemu
```

```
...
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Figure 1.3 – run in xv6: the child becomes echo

Then the child was made to run `ls` by changing two lines:

```
    char *args[] = { "ls", 0 };
    exec("ls", args);                // step 2: become ls
```

Figure 1.4 – The change that makes the child run ls

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
[... 24 entries omitted ...]
run            2 27 35064
pingpong       2 28 35056
console        3 29 0
parent: child 4 ended with status 0
```

Figure 1.5 – run in xv6: the child becomes ls

Finally the exec line was changed to `exec("nosuchprog", args);`, a program that does not exist:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

Figure 1.6 – run in xv6: exec of a missing program

Observations:

1. **The three calls on xv6 and on Linux.** In xv6, `run.c` uses `fork()`, `exec()` and `wait()`; on Linux it used `fork()`, `execlp()` and `waitpid()`. The three steps and their meaning are the same: `fork` copies the process and returns 0 in the child and the child's PID in the parent; `exec` replaces the program of the calling process and returns only if it fails; `wait` blocks until the child ends and gives back its exit status. Only the details differ: xv6's `exec` takes the path and an argument array and does not search a PATH, its `wait` takes only the address of the status, and the status is the plain exit code, without a `WEXITSTATUS` macro.
2. **Who prints "hello from the child".** `echo` prints it. After a successful `exec` the child no longer runs the code of `run`: its memory now holds the `echo` program, which prints its arguments. That is also why `exec failed` is not printed in that case.

3. **Status 1 here and 127 on Linux.** The status is simply the number the child passes to `exit`, and the two programs chose different numbers: after a failed exec, the xv6 `run.c` calls `exit(1)`, while the Linux `run.c` calls `exit(127)`, the number shells use for “command not found”.

2 Part 2. pingpong

The parent sends one byte to the child through a first pipe. The child prints that it received the ping, writes the byte back through a second pipe and exits; the parent reads it, prints that it received the pong and waits for the child. Each process first closes the two pipe ends it does not use.

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);

  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    exit(0);
  }

  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  b = 'x';
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

Figure 2.1 – user/pingpong.c

```
$ pingpong
6: received ping
5: received pong
```

Figure 2.2 – pingpong in xv6

The child (PID 6) prints “received ping” first, then the parent (PID 5) prints “received pong”. PIDs 3 and 4 had been used by `run` in the same session.

Observations:

1. **Why two pipes.** A pipe carries data in one direction only: bytes written into its write end come out of its read end. If both directions used the same pipe, the parent could read back its own “ping” byte before the child reads it, and nothing would tell the two processes whose

byte they are reading. With one pipe for each direction, every byte has exactly one sender and one receiver.

2. **What makes the parent wait.** The `read` call. `read(c2p[0], &b, 1)` blocks until there is a byte in the pipe, so the parent sleeps until the child writes the pong. In the same way the child's `read(p2c[0], &b, 1)` blocks until the parent's ping arrives, which is why "received ping" is always printed first.

The optional `primes.c` was not done.

Conclusions

Running the same program on Linux and inside xv6 showed that the process interface is the same idea in both systems: a process is created with `fork`, turned into another program with `exec`, and collected with `wait`, which returns the number the child passed to `exit`. The differences are only in the library around the calls, such as `execlp` searching the PATH on Linux or the `WEXITSTATUS` macro, while the kernel calls underneath do the same work.

The `pingpong` program showed how processes communicate. A pipe is a one-way channel inside the kernel, so a conversation in both directions needs two pipes, and every process should close the ends it does not use. No explicit synchronisation was needed: because `read` blocks until data arrives, the order of the messages follows automatically from the pipes themselves.